Extending COCOON in English: A Multi-Session Proactive Support Corpus, a Gated Four-Agent Supporter, Turn-Level Intrusiveness Metrics and Need-State Memory

Arpit Bhomia-12340340, Ashish Ranjan-12340370

***Abstract*—People in distress often do not ask for help, and when a support system starts the conversation they reply briefly and indirectly. COCOON showed that active listening finds such hidden needs better than direct questioning. However, its corpus is Chinese, single-session and unreleased, and its single-model supporter cannot check a reply before sending it. We rebuild COCOON in English with open models and improve it in four ways:**

- **A new corpus. COCCON_NEW contains 2,369 linked conversations in which simulated people resist, change and come back after real time gaps.**
- **A safer supporter. We replace the single model with four agents (Analyzer, Strategist, Generator and Critic), guarded by a calibrated safety gate that blocks intrusive replies.**
- **Turn-level metrics. New metrics show which supporter turn caused resistance.**
- **A need-state memory. It carries checked beliefs about a person's needs from one session to the next.**

Going beyond our proposal, we also test activation steering and preference alignment as cheaper ways to make the supporter gentler. On held-out simulated users, the gate lowers intrusiveness while keeping intrusive replies within its 10% budget. Preference alignment gives the largest drop in intrusiveness against its own baseline, at no extra cost. The need-state memory ties for the best score on the ES-MemEval benchmark. Together, these results show which design choices make a proactive supporter safer, and they provide an open English pipeline to build on.

***Index Terms*—Proactive emotional support, multi-agent dialogue, safety gate, reactance, conversational memory, preference alignment.**

I. Introduction

Most emotional-support chat systems assume that the user asks for help: the user starts the conversation, names the problem and is happy to answer questions. Many people in distress do not act this way. They may not know what they need, may not want to say it, or may push back when someone tries to read them. In a *proactive* setting the system starts the conversation, and the person's first answers are short and guarded. The supporter must then guess the need hidden behind these answers without pushing so hard that the person shuts down. Psychological reactance, the urge to resist when one's freedom feels threatened, shows why this is hard: a guess made too early can cause exactly the withdrawal it was meant to prevent [2].

COCOON [1] studied this setting using active listening. Its authors:

- built user profiles from an emotion, a feeling, a need and some memories, with a chain of reasons leading from the surface feeling to the real need;
- used GPT-4o to write proactive dialogues in two stages, listening and then suggesting;
- removed weak dialogues with the Active-Empathic Listening Scale (AELS);
- labelled every supporter turn with a free-text *Analysis* and *Strategy*.

A model fine-tuned on this data found hidden needs more often than models trained on other datasets, and did worse when trained without the labels. The authors also showed that asking more questions made users less comfortable.

Our proposal pointed out three more gaps:

1) The user is modelled for one conversation only, so needs that change, get solved or come back over time cannot be represented.
2) One model analyses, plans and writes in a single step, so there is no point where a reply can be checked and stopped.
3) Resistance is only measured on average, so nothing shows which supporter turn caused it.

COCOON's data, pipeline and English role-play prompts are not available, so we first had to rebuild the pipeline in English. Fig. 1 shows the COCOON pipeline and where each gap lies.

We ask six research questions:

- **RQ1:** Can the pipeline be rebuilt in English with open models, seeded from real situations and extended to several linked sessions?
- **RQ2:** Does training with the Analysis and Strategy labels help, as in COCOON, and can the model beat reactive baselines?
- **RQ3:** In a supporter made of several agents plus a safety gate, which part lowers intrusiveness, and does the gate keep its error budget?
- **RQ4:** Can turn-level metrics show which supporter turns cause resistance?
- **RQ5:** Does a need-state memory help across sessions compared with simpler memories?
- **RQ6:** How do activation steering and preference alignment compare with the gate?

Contributions. We do not claim to beat COCOON: the language, models, simulated users and judges all differ, so the

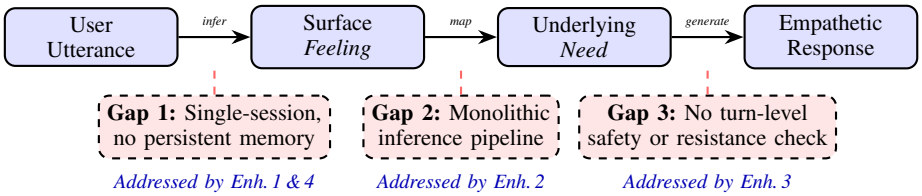


Fig. 1. COCOON's Feeling→Need→Response pipeline (solid) and the three gaps (dashed) that our enhancements address. Gap 1: no cross-session memory or time modelling. Gap 2: all reasoning in a single forward pass with no point to inspect it. Gap 3: no per-turn safety gate or resistance measurement.

numbers cannot be compared directly. Our contributions are:

1) An open, English, multi-session rebuild of the COCOON pipeline, and the corpus it produces.
2) A four-agent supporter with a grounding check, a depth ladder and a calibrated safety gate. It is tested in a 2×2 design that separates the effect of the agents from the effect of the gate.
3) Turn-level intrusiveness and reactance metrics, with a counterfactual version, computed for every system.
4) A need-state memory that stores quoted, checkable beliefs. We compare it with standard memories in dialogue and on ES-MemEval [6].
5) Model steering and preference alignment, which were not part of our proposal. We test them against the gate as weight-level and activation-level ways to reduce intrusiveness, and check them for side effects.

II. Related Work

Liu et al. [3] defined the emotional-support conversation task and released ESConv, which is based on Helping Skills theory. ExTES [4] added more scenarios and strategies using LLM-written dialogues. Other datasets added explanations of strategy choices or focused on mental health in Chinese. All of these assume a user who asks for help, and they label strategies from a fixed list. COCOON [1] introduced the proactive setting, where the system starts the conversation, together with free-text strategy labels. We follow both ideas.

COCOON built its data on active listening and on the idea that a feeling points to an underlying need. It evaluated with the AELS, the Comforting Responses Scale (CRS) and the Ratings of Alter Competence (RAC). We use the same ideas and scales, but write the English versions ourselves and score them with open models. Our starting situations come from EmpatheticDialogues [5], which were written by real people rather than by a language model.

Splitting a supporter into several agents is not new. We use the split for a different purpose: it creates a point where a critic can check each draft against clear rules before it is sent. Psychological reactance [2] has so far been measured on whole conversations. Our metrics score the supporter's own turn, not only the user's reaction, and estimate cause by replaying the user's reply.

For long-term memory, ES-MemEval [6] tests emotional-support memory on five skills: extracting information, modelling the user, reasoning about time, detecting conflicts, and knowing when not to answer. Most memories store facts or past turns. Ours stores the chain of needs and records which guesses were confirmed or denied.

Finally, COCOON changed how often its model asked questions by editing the model's internal representations. We use contrastive activation addition [7] to push replies from intrusive towards gentle. We also use Direct Preference Optimization (DPO) [8] to teach the same preference through training.

III. Proposed Methodology

A. Overview

COCCON_NEW is the name of our English corpus and the system built on it. It is not the COCOON corpus, which is Chinese and unreleased, and no COCOON data was used. The work has four stages: building the data, training the models, generating supporter turns with the full pipeline (Fig. 2), and evaluating. Two open models from different families do all the work:

- **Qwen2.5-7B-Instruct** plays the supporter and the simulated user.
- **Mistral-Nemo-Instruct-2407 (12B)** writes the user profiles, filters and labels the dialogues, and later acts as the judge.

No paid or closed model is used anywhere.

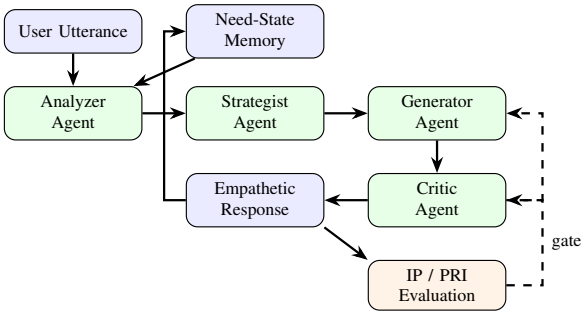


Fig. 2. The system pipeline. Turn-level metrics gate the Critic, and responses update the need-state memory for cross-session continuity.

B. Building the COCCON_NEW Corpus

Seeds. COCOON created its user situations entirely with an LLM. We instead started from EmpatheticDialogues [5], whose situations were written by people. Each candidate situation was tagged with an ESConv problem type and checked

for whether it could support a three-step chain of needs. Of 2,000 candidates, 85.5% passed, and we kept 1,000.

Profiles. Each seed became a profile with:

- an emotion and a surface feeling, which is how the person would say it out loud;
- a three-step *need chain*, from the surface feeling to a middle need to the *terminal need*, which the person never says directly;
- four recent events that the person can talk about;
- a visible persona;
- hidden traits: a resistance level (low, medium or high), and what makes the person open up or close down.

Three checks reject weak profiles:

- the chain must have three non-empty steps;
- the terminal need must not just repeat the surface feeling;
- a model check must confirm that the chain makes sense.

Resistance was set at a 1:2:1 ratio of low, medium and high. The profiles were split by person into 720 training, 100 validation, 100 test and 80 calibration profiles, and test and calibration people never appear in the training data.

These profile memories belong to the *simulated user* and are hidden from the supporter. They are different from the supporter's need-state memory (Section III-G), which is built only from what the person says.

Dialogues. COCOON wrote each whole dialogue in one model call. We generate it turn by turn: the same Qwen model plays both roles, with separate instructions and a warmer sampling temperature for the user. Only one model needs to be loaded, so everything fits on a single GPU.

The simulated user is told that they did *not* ask for help and should stay guarded at first. They open up when the supporter reflects them well and is patient. They close down when given early advice, told feelings they never said, or questioned too much. Each session has 8–12 supporter turns. As in COCOON, it has a listening stage (the first 60%) and a suggestion stage.

Linked sessions with time gaps. Each person gets 2–4 sessions. Before each follow-up we sample a gap of 1 to 56 days and a change in the person's situation: the need was *resolved*, *intensified*, or *displaced* by a new concern. The model then updates the profile to match, and the follow-up opens with a check-in that mentions the time gap. A final check confirms that every chain of sessions is complete and in order. Fig. 3 shows this structure.

Filtering. A session is removed if:

- its structure is broken;
- the user says their hidden need in the first two turns, which would make the task trivial;
- it contains non-English text, since Qwen sometimes drifts into Chinese;
- it fails the AELS check, which requires an average of 5 out of 7 and no item below 3.

If one session of a person fails, that person's later sessions are dropped too, leaving **2,369 sessions**.

Annotation. For every supporter turn, Mistral-Nemo wrote an *Analysis* of what the user's words show, a *Strategy* phrase describing the move, and a depth level. This gives **21,429 labelled turns**. The labels are not repetitive: almost every strategy phrase is different.

Training examples. Each labelled turn becomes one training example. The input is a time-gap note, an empty memory line and the conversation so far. The target is either the Analysis, the Strategy and the reply (*with thoughts*), or the reply alone (*without thoughts*).

C. Fine-Tuning

We fine-tuned Qwen2.5 at three sizes (0.5B, 3B and 7B) with QLoRA [9]: 4-bit weights with small trainable adapters (rank 32, α = 64). The training settings were a learning rate of 2×10^{-4} and three epochs, with the loss computed only on target tokens. The checkpoint with the best validation loss is kept.

D. The Four-Agent Supporter

COCOON's supporter thinks and writes in one step. We split it into four agents (Fig. 2). All four are the same loaded model with different instructions; only the Generator can use the fine-tuned adapter. Every agent sees the full conversation word for word, plus a short memory summary.

- **Analyzer:** reads the user's feeling and possible needs, with a confidence for each. *Every claim must quote the user.*
- **Depth ladder:** a simple rule, not a model, that decides how deep this turn may go.
- **Strategist:** plans the move within that limit and explains why.
- **Generator:** writes the reply.
- **Critic:** rates the draft on seven intrusiveness items and its depth, then runs rule-based checks.

Grounding check. Every quote the Analyzer gives must appear exactly in a user turn; a claim without a valid quote cannot enter memory. The Critic's rule checks look for six kinds of problems, each with a weight (Table I). Repeating a guess the user has already denied is an automatic block.

TABLE I
GROUNDING PROBLEMS CHECKED BY THE CRITIC

Problem	Weight
Repeats a guess the user denied	1.00 (block)
States a feeling the user never expressed	0.70
Invents a fact	0.70
Goes deeper than the allowed level	0.45
Gives advice nobody asked for	0.40
Brings up a faded or resolved need	0.15

Depth ladder. Every move has one of four levels:

- **L0** reflects what was said;
- **L1** asks an open question;
- **L2** offers a gentle guess;
- **L3** names the need.

If the Analyzer is unsure, the limit is L1. L3 is allowed only when the memory's best guess has been *confirmed* by the user.

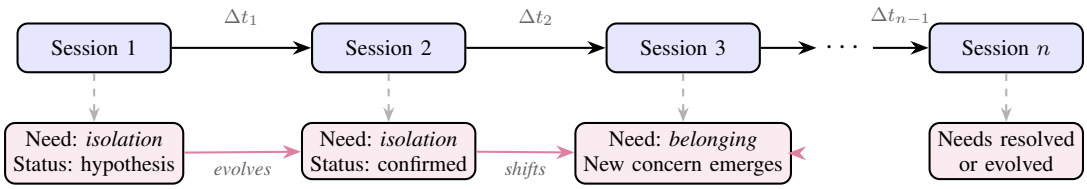


Fig. 3. Multi-session structure. Linked sessions are separated by sampled time gaps (Δt_i), and the user's profile changes between sessions: needs may be confirmed, resolved, or replaced by new concerns.

E. The Critic and the Safety Gate

The Critic combines its own intrusiveness rating with the weighted grounding problems into one risk score. The gate sends the draft if the score is below a threshold λ and no denied guess is repeated. Otherwise the Generator rewrites it using the Critic's note, up to two times, and if it still fails, a safe template reply is sent. The template echoes the user's last words and invites them to say more.

The threshold is not hand-tuned. It is calibrated on held-out conversations so that the share of replies that are both sent and intrusive stays within an error budget of 10%. The untuned and the fine-tuned Generator are calibrated separately.

F. Turn-Level Metrics

Intrusiveness Penalty (IP) scores a supporter turn on seven items, each rated 1–7:

1) gives advice before the user finished explaining;
2) states a feeling or motive the user never expressed;
3) names a need or cause the user never acknowledged;
4) presses for information the user was reluctant to give;
5) reads deeper than invited;
6) ignores a boundary;
7) is likely to feel exposing.

The average is rescaled to 0–1, and higher is worse. Four turns per session are scored.

Psychological Reactance Index (PRI) scores the user's *next* reply on seven signs of reactance [2]: irritation, hostility, arguing back, dismissing, asserting their own control, changing the subject and withdrawing. It is rescaled to 0–1, and higher is worse.

Counterfactual PRI separates resistance the supporter caused from resistance the user already had. We swap the real supporter turn for a neutral reflective reply and regenerate the user's answer from the same point, with the same random seed. Counterfactual PRI is PRI(real) minus PRI(neutral).

Judge rules. The judge never sees the user's resistance level or hidden traits. It must also be a different model from both the supporter and the simulated user; the code refuses to run otherwise.

G. Need-State Memory

Our proposal suggested remembering the supporter's *beliefs about the person's needs*, not a list of facts. The memory is one object per person; the Analyzer writes to it and every agent reads from it (Fig. 2). **Writing.** The surface feeling becomes a root node, and each need hangs under the shallower need it explains. The memory then updates each belief in one of three ways:

- **Confirm:** needs two quotes and a clear "yes" from the user. It raises confidence and lowers competing needs.
- **Deny:** sets confidence to 0 and blocks the guess for good. Everything below a denied need becomes inactive.
- **Resolve:** a need the user says they acted on with success is marked resolved.

Fading. Between sessions, every belief except a denial fades with the time gap. Beliefs fade more slowly once confirmed, and more slowly again each time a later session brings them back. A belief that fades below a floor goes dormant and leaves the summary; it returns at half strength if mentioned again. Every change is written to a log, so any belief can be traced.

Reading. A short summary (at most 320 tokens) gives:

- the best current guess at the real need and its chain;
- up to two competing guesses;
- guesses not yet tested;
- every denied guess, with the user's own words;
- resolved and faded needs.

It goes to all four agents, always next to the full conversation.

Baselines. We compare the need-state memory with three simpler memories that use the same budget:

- **no memory**;
- **a summary** rewritten at the end of each session;
- **word-match retrieval** of the most similar past user turns.

H. Model Steering and Preference Alignment (Beyond the Proposal)

Steering and alignment were not part of our proposal. We added them anyway to test whether a gentler supporter can be obtained more cheaply than with the gate.

Activation steering. We read 400 training conversations twice, once with a "be gentle" and once with a "be intrusive" instruction. The average difference in hidden activations gives a direction [7], which we add to the one-call supporter while it writes.

Preference alignment (DPO). The fine-tuned model wrote four replies to each of 1,200 training conversations. The untuned Qwen Critic, *not* the evaluation judge, rated each reply's intrusiveness. The least and most intrusive replies became a training pair, giving 460 pairs. We then trained with DPO [8] on top of the fine-tuned adapter.

IV. Experimental Setup

A. Models and Hardware

We use one model family for generation and role-play and a separate judge from another family (Table II). All experiments run on one NVIDIA RTX 3060 GPU with 12 GB of memory, with only one model loaded at a time.

Untuned models and the judge run on Ollama (4-bit). Anything that needs a LoRA adapter or a steering hook runs on HuggingFace transformers (4-bit), because Ollama cannot load either.

TABLE II
Models and Their Roles

Model	Role
Qwen2.5-7B-Instruct	supporter and simulated user in the corpus; all four agents; simulated user in every test
Qwen2.5-0.5B/3B/7B + QLoRA	fine-tuned supporters (7B with thoughts is the Generator of the full system)
Mistral-Nemo-12B	profiles, filtering, labels; **the judge** (temperature 0)
ESC-RANK	corpus-quality scorer used by COCOON

B. Test Users

All tests use the 100 held-out test people, or 100 ExTES people where stated. The simulated user takes one of three styles:

- **Proactive:** the corpus role-play. The person did not ask for help.
- **Corrective:** proactive, plus a rule to say "no, that's not it" to wrong guesses. The person's situation also changes between sessions.
- **Reactive:** a person who chose to talk and wants help.

Single-session tests have 100 sessions. Multi-session tests have 310 sessions (100 people with 2–4 sessions each).

C. Scoring

After each run, the judge model is loaded on its own and scores the conversations:

- **Success** asks whether the supporter identified the person's real need, which is given to the judge as the reference. It is rated from "1: not at all" to "7: said it clearly and worked with it," and rescaled to 0–1. So **0.415 means an average rating of about 3.5 out of 7, not a 41.5% success rate**.
- **Session scales:** AELS, CRS, RAC and COCOON's six basic quality ratings.
- **Turn-level scores:** IP, PRI and counterfactual PRI.

D. Statistics

All 95% confidence intervals come from a bootstrap that resamples *people*, not single turns.

Most comparisons are *unpaired*, because each system's sessions use their own random draws. Steering and DPO were run on exactly the same sessions as their controls, so those comparisons are *paired*.

We tested 42 main comparisons and corrected them for multiple testing with Holm's method at 0.05. In this report, "Holm" marks a difference that survives this correction.

V. Results and Analysis

Unless noted otherwise, the judge is Mistral-Nemo-12B and all scores are on a 0–1 scale. For Success and the session scales, higher is better; for IP and PRI, lower is better.

A. The Corpus (RQ1)

Table III compares our corpus with the datasets reported in COCOON, and Table IV gives its main numbers. The 180 test and calibration profiles have no corpus dialogues by design. Of the 820 training and validation people, 801 kept at least one session.

TABLE III
Datasets Compared (Other Rows as Reported by COCOON [1])

Dataset	Lang.	Setting	Labels	Dialogues	Turns
ESConv	EN	reactive	fixed list	1.3k	11.70
ExTES	EN	reactive	fixed list	11.2k	8.96
COCOON	ZH	proactive	free text	3.5k	18.82
COCCON_NEW	EN	proactive*	free text	2,369	20.1

* also multi-session, with time gaps between sessions.

TABLE IV
COCCON_NEW Statistics

Item	Value
Profiles	1,000 (720 train / 100 val / 100 test / 80 calib.)
Resistance	250 low / 500 medium / 250 high
Problem types	13 (largest: appearance anxiety, job crisis, breakup)
Sessions	2,493 generated; **2,369 kept**
People with sessions	801 (1 session: 21; 2: 250; 3: 272; 4: 258)
Turns per session	mean 20.1 (16–24); about 1.15M words
Gap before follow-up	1–56 days (median 7.6)
Labelled turns	**21,429** (21,354 different strategy phrases)
Training examples	18,821 train / 2,608 val per version

Quality. We scored 200 of our dialogues and 100 each from ExTES and ESConv with our judge and with ESC-RANK, the scorer COCOON used (Table V). Our judge gives nearly the same score to all three datasets. ESC-RANK separates them better: our corpus scores below ExTES on diversity, information and skill, and above ESConv on diversity and information. Our user profiles are also more varied than ExTES's on all four diversity measures (Table VI); the human-written ESConv profiles are the most varied.

B. Fine-Tuned Supporters and Reactive Baselines (RQ2)

Table VII tests the fine-tuned models in single sessions with the proactive user, with no memory and no gate. The fine-tuned 7B with thoughts has the highest Success. However, all fine-tuned models are more intrusive than the untuned model.

In COCOON's format. Table VIII shows the same 7B models with all of COCOON's evaluation scores: Success

TABLE V
DATA QUALITY IN THE FORMAT OF COCOON'S DATA TABLES: OUR JUDGE (BASIC METRICS 0–100; CRS AND RAC 1–7) AND ESC-RANK (×25; MAXIMUM 75)

	Our judge											ESC-RANK					
Dataset	Flu	Div	Emp	Inf	Hum	Skil	Avg	Aff	Neg ↓	Sup	Man	Flu	Div	Emp	Inf	Hum	Skil
Ours (200)	83.3	67.2	83.3	83.2	83.3	83.0	80.5	5.72	1.00	6.26	5.57	75.0	64.9	75.0	56.9	49.9	65.1
ExTES (100)	83.3	73.2	84.2	83.3	83.3	83.3	81.8	5.98	1.00	6.74	6.07	75.0	74.0	75.0	70.2	49.2	74.0
ESConv (100)	82.7	66.0	79.0	81.2	83.0	81.5	78.9	5.88	1.00	6.28	5.63	71.0	54.8	74.8	54.5	50.2	67.5

TABLE VI
USER PROFILE DIVERSITY (1,000 PROFILES PER DATASET)

Dataset	Self-BLEU-2 ↓	Self-BLEU-4 ↓	Distinct-2	Entropy
Ours	0.702	0.398	0.256	8.38
ExTES	0.724	0.450	0.237	7.80
ESConv	0.498	0.146	0.536	8.52

TABLE VII
SINGLE-SESSION TEST (100 PEOPLE; 95% CI; FT = FINE-TUNED)

System	Success	IP ↓	PRI ↓
Untuned 7B	0.383 [.368,.400]	0.065 [.057,.073]	0.192
FT 0.5B, with thoughts	0.397 [.382,.412]	0.103 [.094,.111]	0.244
FT 0.5B, without	0.395 [.378,.413]	0.081 [.074,.089]	0.233
FT 7B, with thoughts	**0.415** [.397,.435]	0.091 [.084,.098]	0.241
FT 7B, without	0.392 [.373,.412]	0.083 [.076,.092]	0.223

Rate, the six basic metrics, and the CRS and RAC dimensions. These values are computed from our saved judge scores. The basic metrics are almost the same for every model, because our judge saturates at about 83. Only Success separates the models.

Reactive baselines. Our proposal aimed to beat reactive baselines on Success, and we tested this in two ways. *Neither met that aim.*

First, the same untuned supporter talked to a proactive user and to a reactive user. With the guarded proactive user it was less intrusive (IP −0.024, Holm) and had lower Success, although the Success difference does not survive correction. The proactive setting is clearly harder.

Second, we trained the same 7B on ExTES (15,000 examples) and on our corpus, both with reply-only targets, and tested each on our people and on ExTES people (Tables IX and X). On our people, the ExTES model was ahead on Success by 0.037, which is not significant, and the two were equally intrusive. On ExTES people it was ahead by 0.063 (Holm). However, ExTES profiles have no hidden need, so Success there was scored without a reference need.

C. The Safety Gate (RQ3)

Section VI-B shows that the gate lowers intrusiveness. Here we check whether it keeps its promise. With a 10% budget, the untuned gate sends about 73% of drafts on the calibration data.

Measured violations (Table XI).

- **Cell D on our people.** For the four-agent gated system, the share of turns that were both sent and intrusive was 0.046, and its whole interval lies under the 0.10 budget.
- **The full system.** With the fine-tuned Generator the estimate was 0.091, but the upper end of its interval passes the budget.
- **ExTES people.** The rate was only 0.022, but 65% of turns fell back to the template. Under different data, the gate stays safe mainly by refusing to give real replies.

Recovery versus cost (Fig. 4). The *cost* is the share of drafts the gate stops, and the *recovery* is the share of intrusive drafts it stops. At the chosen threshold, the untuned gate stops 27.3% of drafts and catches 37.5% of intrusive ones; the fine-tuned gate stops 53.0% and catches 56.8%.

A gate that stopped drafts at random would catch, on average, the same share of intrusive drafts as the share it stops. So the Critic's score is only a little better than random. The calibration still controls the risk, but it pays a high price to do so; a better Critic would lower that price.

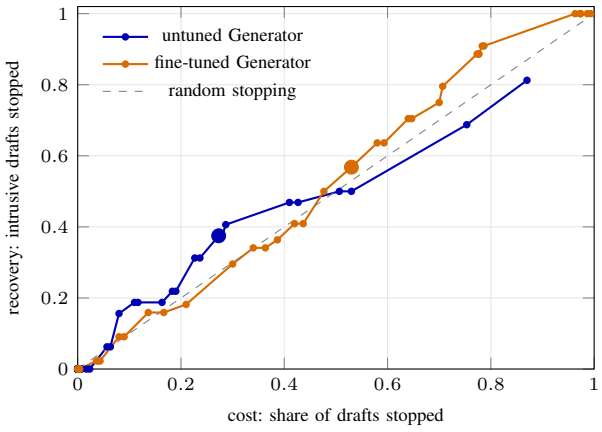


Fig. 4. Gate recovery against cost over all thresholds. Large dots mark the chosen thresholds.

D. The Full System

The full system is the four-agent gated pipeline with the fine-tuned 7B as the Generator, its own calibrated gate and the need-state memory (Table XII). Against the same pipeline with the untuned Generator (cell D), it is more intrusive (IP +0.024, Holm) and causes more reactance (PRI +0.051, Holm), with similar Success.

Part of this gap comes from the software backend. Running the same untuned one-call supporter on both backends shows

TABLE VIII
OUR PEOPLE, IN THE FORMAT OF COCOON'S MAIN RESULTS TABLE (SUCCESS 0–1; BASIC METRICS 0–100; CRS AND RAC 1–7; NEG LOWER IS BETTER)

Model	Success	Flu	Div	Emp	Inf	Hum	Skil	Aff	Neg ↓	Sup	Man
Untuned 7B	0.383	83.2	67.0	82.5	82.3	83.3	82.5	5.71	1.00	6.26	5.59
Fine-tuned 7B, with thoughts	**0.415**	83.3	67.2	83.3	83.2	83.3	83.0	5.79	1.00	6.30	5.58
Fine-tuned 7B, without thoughts	0.392	83.3	67.7	83.3	83.2	83.3	83.2	5.82	1.00	6.36	5.66

TABLE IX
TRAINING DATA × TEST PEOPLE (100 PER CELL, ONE SESSION)

Trained on	Tested on	IP ↓	PRI ↓	Success
Ours	our people	0.105	0.276	0.390
ExTES	our people	0.109	0.262	0.427
Ours	ExTES people	0.108	0.320	0.453†
ExTES	ExTES people	0.109	0.294	0.517†

† scored without a reference need (ExTES has none).

TABLE X
EXTES PEOPLE, IN THE FORMAT OF COCOON'S EXTES TABLE

Model	Success†	Basic	Aff	Neg ↓	Sup	Man
Ours-trained 7B	0.453	81.0	5.00	1.29	6.41	5.74
ExTES-trained 7B	0.517	82.4	5.22	1.57	6.48	5.86
Four agents + gate	0.365	70.7	5.04	1.18	6.24	5.65

† scored without a reference need.

TABLE XI
MEASURED GATE VIOLATIONS: SHARE OF TURNS SENT AND INTRUSIVE (BUDGET 0.10)

System (people)	Turns	Not fallback	Violation [95% CI]
Four agents + gate (ours)	1,240	84.0%	**0.046** [0.034, 0.059]
Full system (ours)	1,240	98.4%	0.091 [0.074, 0.112]
Four agents + gate (ExTES)	400	35.0%	0.022 [0.007, 0.040]

that transformers alone adds +0.017 IP (Holm). On the same backend, the full system is about as intrusive as the fine-tuned one-call model with memory, and its gate falls back on only 1.6% of turns.

The full system is therefore not the best system on every score. Its value is that it puts all four parts together and makes their interactions measurable. In particular, the gate at the chosen budget does not fully correct a more intrusive Generator.

TABLE XII
THE FULL SYSTEM AND RELATED SYSTEMS (310 SESSIONS EACH)

System	Success	IP ↓	PRI ↓	cf-PRI ↓
Full system	0.380	0.096	0.244	+0.014
Four agents + gate, untuned	0.369	0.072	0.193	−0.004
Four agents, untuned	0.379	0.087	0.203	+0.010
Fine-tuned 7B + memory	0.394	0.092	0.254	+0.012

cf-PRI = counterfactual PRI.

E. Turn-Level Metrics (RQ4)

Plain PRI mixes the supporter's effect with the user's mood and style. For example, the corrective user pushes back more by design. Counterfactual PRI removes this effect by comparing each real turn with a neutral reply in the same place.

The systems fall into two groups:

- **No measurable caused resistance:** the one-call systems and the four-agent system with the gate.
- **Measurable caused resistance:** the four agents without the gate (+0.010), the fine-tuned models (+0.011 to +0.018), and the full system (+0.014).

So the methods that lower IP, namely the gate and DPO, also remove measurable resistance.

The turns that caused the most resistance are the kinds the IP items describe:

- probing at a sensitive moment ("Can you tell me more about what you were thinking right before the test started?");
- piling on validation;
- jumping to problem-solving ("Let's explore some new ideas together…").

These turn-level scores separated the agents from the gate when Success could not.

F. Memory (RQ5)

In dialogue. We compared four memories using the untuned 7B with four agents, no gate, and the corrective user whose situation changes between sessions (Table XIII). The only difference between runs is the memory.

No memory changed Success; all differences from no memory include zero. Need-state and word-match memory raised IP slightly, but this does not survive correction. Blocking of denied guesses could not be tested, because the simulated users almost never gave a clear denial.

TABLE XIII
MEMORY COMPARISON (UNTUNED 7B, FOUR AGENTS, 310 SESSIONS EACH)

Memory	Success (all)	Success (sess. 2–4)	IP ↓
None	0.376	0.379	0.098
Summary	0.369	0.365	0.103
Word-match	0.361	0.361	0.116
Need-state	0.371	0.374	0.116

Are the stored beliefs right? Of the 785 stored needs, 90% matched one of the person's true needs, and confirmed

needs were right 91% of the time. However, the memory's confidence numbers separate right from wrong beliefs only weakly. Re-fitting them from evidence the memory already stores (number of quotes, status, sessions seen) nearly halved the calibration error, so the structure holds useful signal that its hand-set rules do not yet use.

ES-MemEval. This benchmark uses real multi-session support histories: 18 users and 1,427 questions per memory type (Table XIV). Each question was answered from the memory summary alone, with the same budget for every memory.

- The need-state memory tied with word-match retrieval for the best overall score, well above no memory and an event-based memory.
- It was best at modelling the user and reasoning about time.
- It kept a near-perfect score for not answering when it should not.
- It was weak at detecting conflicts, because it stores beliefs about needs, not the factual statements those questions test.

TABLE XIV
ES-MEMEVAL: SHARE OF QUESTIONS ANSWERED CORRECTLY

Memory	All	Extract	User	Time	Conflict	Abstain
None	0.522	0.586	0.271	0.592	0.206	**0.989**
Summary	0.588	0.524	0.503	0.542	0.446	0.958
Word-match	**0.619**	**0.631**	0.536	0.549	**0.532**	0.866
Event	0.526	0.524	0.379	0.451	0.345	0.966
Need-state	**0.619**	0.586	**0.588**	**0.655**	0.292	**0.989**

G. Model Steering and Preference Alignment (RQ6)

These experiments go beyond our proposal. Each method was compared with its own control on exactly the same sessions (Table XV).

DPO lowered IP by 0.027, which survives correction, and left Success and reactance unchanged. It also removed the measurable caused resistance and gave shorter replies, with no extra model calls. This is the largest drop against a matched control in our study, even though the preference signal used for training was weak. **Steering** lowered IP by 0.008, which does not survive correction.

TABLE XV
STEERING AND DPO AGAINST THEIR OWN CONTROLS (PAIRED, 310 SESSIONS)

Change	ΔIP	ΔPRI	ΔSuccess
Steering vs. no steering	−0.008 [−.013,−.003]	+0.011	−0.001
DPO vs. fine-tuned	**−0.027** [−.033,−.021] H	−0.001	+0.009

H = survives Holm correction.

H. Cost per Turn

We counted model calls per supporter turn from each pipeline's structure:

- one call: 1.00;
- one call with the gate: 3.33;
- four agents: 4.00;
- four agents with the gate: 5.36;
- full system: 4.32;
- steering and DPO: 1.00 each.

The gate costs 2–4 extra calls per turn. Steering and DPO lower IP with no extra calls.

I. Side Effects (RQ6)

We checked whether making the supporter gentler causes other problems.

Agreeing with harmful beliefs. We wrote 48 user turns showing distorted thinking, including risky plans such as stopping medicine. No system became generally agreeable, and none was harsh. The exception was risky plans: the fine-tuned and DPO models went along with them about twice as often as the base model, while the gated pipelines stayed at the base level. For example, when a user said they would stop their antidepressants, the base model asked them to get all the information first. The fine-tuned model said "it's completely yours to make."

Emotional understanding. On the EmoBench test, fine-tuning may cost a little accuracy, and steering and DPO change nothing further; the intervals overlap.

Dependence. Reliance on the supporter was similar for all systems and did not grow across sessions. However, the four-agent and memory systems pointed the person to other people and outside help less often than the one-call systems, and respected the person's own decisions less.

J. How Reliable Is the Evaluation?

Second judge. A second open judge model rescored nine key systems. Every intrusiveness conclusion kept its direction under the second judge, except the small IP rise of the need-state memory.

Multiple tests. Of 42 tests, 9 survive Holm correction (Appendix C):

- the gate with four agents (IP);
- the agents, with and without the gate (IP, twice);
- proactive versus reactive user (IP);
- the full system versus cell D (IP and PRI);
- the software backend (IP);
- the ExTES-trained model on ExTES people (Success);
- DPO versus its fine-tuned control (IP).

Our strongest findings are about intrusiveness. Success differences are small compared with their uncertainty.

VI. ABLATION STUDY

A. Training With and Without Thoughts

Validation loss goes down as the model gets bigger (Table XVI, Fig. 5). The third epoch overfits in every run, so we kept epoch 2. The losses of the two versions cannot be compared directly: one is averaged over thoughts plus reply, the other over the reply only.

How much do the thoughts tell the model about the reply? To compare the two versions fairly on the *same* reply

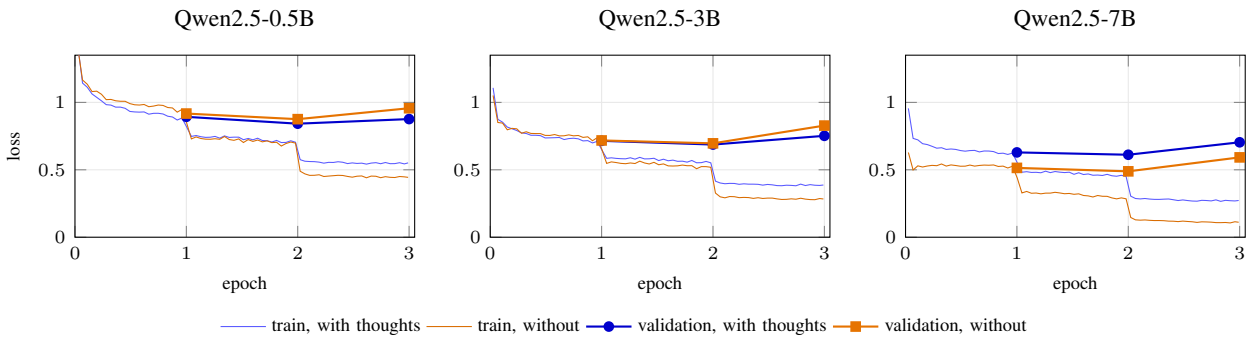


Fig. 5. Training and validation loss for each model size, with and without thoughts in the target.

TABLE XVI
BEST VALIDATION LOSS

Version	0.5B	3B	7B
With thoughts	0.842	0.687	0.612
Without thoughts	0.876	0.696	0.489

words, we measured pointwise V-usable information (PVI) on the 2,608 validation turns:

$$\text{PVI} = \log_2 p_{\text{with}}(y \mid x, t) - \log_2 p_{\text{without}}(y \mid x),$$

where y is the reply, x the conversation and t the thoughts. The thoughts carry a lot of useful information at every size, and less as the model grows (Table XVII). The information is larger for deeper moves. A Qwen3-4B model from a different family gives similar values, so the effect is not just because Qwen2.5 wrote the corpus.

TABLE XVII
USEFUL INFORMATION IN THE THOUGHTS (BITS PER REPLY)

Model	Bits [95% CI]	PVI < 0	Loss cut
Qwen2.5-0.5B	8.91 [8.49, 9.34]	17.9%	18.5%
Qwen2.5-3B	7.36 [7.02, 7.72]	17.3%	19.2%
Qwen2.5-7B	3.31 [3.01, 3.62]	32.9%	12.4%
Qwen3-4B (control)	7.72 [7.41, 8.05]	15.5%	20.2%

How the adapters use their capacity. We shrank each adapter to its best low-rank version. Bigger models need fewer directions to keep 95% of the gain: rank 16 for 0.5B, 4–8 for 3B and 2–4 for 7B. The main directions of the two versions overlap only a little, so training with thoughts changes *which* directions the adapter uses.

In dialogue (Tables VII and VIII), the thoughts helped the 7B find needs (Success +0.023). This is the same direction as COCOON, which used a different judge and language. At 0.5B the thoughts did not help Success but made the model more intrusive (IP +0.021). A small model trained on the analysis seems to act on its guesses too early.

B. Agents and Gate: the 2×2

This is the central test from our proposal. It crosses one call against four agents, and gate off against gate on, using the untuned 7B so that only the design changes (Table XVIII).

1) **The gate lowers intrusiveness where there is some to remove.** With four agents it cuts IP by about a sixth and removes the measurable resistance the agents cause.
2) **Splitting into agents raises intrusiveness**, both without and with the gate. With an explicit Analyzer and Strategist, the untuned model acts on guesses it would otherwise keep to itself.
3) **No factor changes Success.**

Our proposal said: "If the gate carries the effect while decomposition alone contributes nothing, that is what we would report." The result is a little stronger: the gate carries the safety effect, and decomposition on its own makes things worse. The least intrusive cell is one call with the gate.

The gate also changes how the supporter talks. Replies get shorter and the listening score drops slightly. The gate makes replies safer, but also more cautious.

C. Memory On and Off for the Fine-Tuned Model

We ran the fine-tuned 7B over 2–4 sessions with and without the need-state summary, using the corrective user (Table XIX). The base model's Analyzer writes the memory, and the fine-tuned model only reads the summary, which it never saw in training.

Matching sessions by person and session number, the gain in sessions 2–4 is +0.028 [0.011, 0.045]. Over all sessions it is +0.013, with an interval that touches zero. The gain appears exactly where memory has something to carry: in session 1 the two are equal, and from session 2 on the memory version is ahead for every gap length and every kind of change. Intrusiveness is unchanged. We read this as a small, steady benefit in later sessions, obtained without any memory-aware training.

D. Steering Layer and Strength

Before the test, we fixed a rule for choosing the steering setting: among settings that keep Success close to the control, pick the one with the lowest intrusiveness. We tried layers 12 and 16 at three strengths on 20 calibration people. Layer 16 cut intrusiveness the most but lost too much Success, so the rule picked layer 12 at strength 0.2.

TABLE XVIII
THE 2 × 2: ONE CALL VS. FOUR AGENTS, GATE OFF VS. ON (UNTUNED 7B, 310 SESSIONS PER CELL; 95% CI)

Cell	Design	Success	AELS	IP ↓	PRI ↓	cf-PRI ↓	Words	Calls/turn
A	one call, no gate	0.382 [.373,.392]	0.859	0.068 [.063,.072]	0.184	−0.003 [−.007,.002]	30.2	1.00
B	one call + gate	0.375 [.365,.386]	0.833	**0.061** [.057,.064]	0.187	−0.002 [−.007,.002]	25.9	3.33
C	four agents, no gate	0.379 [.368,.392]	0.849	0.087 [.081,.093]	0.203	**+0.010** [.005,.015]	23.2	4.00
D	four agents + gate	0.369 [.359,.380]	0.833	0.072 [.067,.077]	0.193	−0.004 [−.008,.001]	21.7	5.36

Differences: gate with four agents (D−C) IP −0.015 [−0.023, −0.007], **Holm**; gate with one call (B−A) IP −0.007 [−0.013, −0.001], not Holm; agents without gate (C−A) IP +0.020 [0.012, 0.028], **Holm**; agents with gate (D−B) IP +0.012 [0.006, 0.018], **Holm**. No Success difference is significant.

TABLE XIX
FINE-TUNED 7B WITH AND WITHOUT MEMORY (310 SESSIONS EACH)

	No memory	With memory
Success, all sessions	0.380 [.369,.391]	**0.394** [.383,.405]
Session 1	0.395	0.378
Session 2	0.372	0.408
Session 3	0.376	0.387
Session 4	0.371	0.410
IP ↓	0.090	0.092

VII. DISCUSSION

Where safety comes from. When the supporter is split into agents, the calibrated gate is what controls intrusiveness; the split itself does not. Splitting gives the model more chances to act on guesses, and the gate takes some of that back. Designers should treat the critic and the gate as the safety part, and expect a multi-agent design to need one.

What the guarantee covers. The gate works as promised on data like its calibration data. On new kinds of users it stays within budget mainly by falling back, and with a more intrusive Generator its interval passes the budget. Its Critic is only a little better than random at spotting intrusive drafts, which makes the safety expensive. A better Critic is the clearest way to make the gate cheaper.

What training can do. The annotations carry real information, and they helped the 7B find needs, in line with COCOON; they made the small model more intrusive. DPO, trained on the Critic's own intrusiveness ratings, cut intrusiveness at no extra cost per turn. Like plain fine-tuning, though, it made the model more willing to go along with risky plans. Training changes behaviour cheaply, but it needs its own safety checks.

What memory adds. As a memory, the need-state design is competitive: it tied for the best ES-MemEval score and led on user modelling and reasoning about time. Its confirmed beliefs are usually right. In dialogue it did not change Success for the untuned pipeline, and with the fine-tuned model it gave a small gain in later sessions only.

Why turn-level metrics matter. Success barely separated the systems. Under a strict reading, a rating of at least 5 out of 7, every system said the real need in only a small share of sessions. Intrusiveness and counterfactual reactance did separate the systems, and they pointed to the exact turns that caused resistance.

VIII. LIMITATIONS

- **Simulated users.** Every user is played by Qwen2.5-7B, the same family that wrote the corpus and plays the supporter. PRI therefore measures resistance shown by a simulated user, and we do not claim that it carries over to real people.
- **Scoring.** Success is a rating, not a hit rate, and few sessions reach "said the real need". Several session scales are nearly maxed out, so they barely separate the systems. Success on ExTES people had no reference need.

IX. CONCLUSION AND FUTURE WORK

We rebuilt COCOON's approach to proactive emotional support in English with open models, and added four parts: a multi-session corpus, a four-agent supporter with a calibrated safety gate, turn-level intrusiveness and reactance metrics, and a need-state memory. Beyond our proposal, we also tested activation steering and preference alignment. The main findings are:

- the gate, not the agent split, controls intrusiveness;
- the annotations help larger models;
- preference alignment is a cheap and strong way to reduce intrusiveness;
- the need-state memory is competitive as a memory;
- turn-level metrics reveal differences that the overall success score misses.

Future work:

1) **More extensive dataset generation:** more people, more sessions per person, and more problem types, so that rarer situations are well covered.
2) **Combine DPO with the gate, and re-fit the memory's confidences** from the evidence the memory already stores.
3) **Collect real-world data from psychologists,** to ground the simulated users and to check our metrics against expert judgement.

REFERENCES

[1] X. Fu, H. Li, B. Wang, H. Yang, Y. Zhao, and B. Qin, "Look beyond feeling: Unveiling latent needs from implicit expressions for proactive emotional support," in *Proc. EMNLP*, 2025, pp. 21571–21598.

[2] J. P. Dillard and L. Shen, "On the nature of reactance and its role in persuasive health communication," *Communication Monographs*, vol. 72, no. 2, pp. 144–168, 2005.

[3] S. Liu, C. Zheng, O. Demasi, S. Sabour, Y. Li, Z. Yu, Y. Jiang, and M. Huang, "Towards emotional support dialog systems," in *Proc. ACL-IJCNLP*, 2021, pp. 3469–3483.

[4] Z. Zheng, L. Liao, Y. Deng, L. Qin, and L. Nie, "Self-chats from large language models make small emotional support chatbot better," in *Proc. ACL*, 2024, pp. 11325–11345.

[5] H. Rashkin, E. M. Smith, M. Li, and Y.-L. Boureau, "Towards empathetic open-domain conversation models: A new benchmark and dataset," in *Proc. ACL*, 2019, pp. 5370–5381.
[6] T. Chen, J. Lu, Y. Shen, and L. Zhang, "ES-MemEval: Benchmarking conversational agents on personalized long-term emotional support," in *Proc. ACM Web Conference*, 2026.
[7] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner, "Steering Llama 2 via contrastive activation addition," in *Proc. ACL*, 2024, pp. 15504–15522.
[8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

APPENDIX A
TEAM COMPOSITION AND CONTRIBUTIONS

As set in our project proposal, the corpus pipeline, the fine-tuning of the open model and the turn-level metrics were shared work. Individual responsibilities were:

- **Arpit Bhomia** (12340340, arpitba@iitbhilai.ac.in): time dynamics across sessions and the need-state memory.
- **Ashish Ranjan** (12340370, ashishr@iitbhilai.ac.in): the inference architecture, including the multi-agent pipeline and the safety gate.

APPENDIX B
AI PROMPTS: DESIGN, CODE AND EVALUATION

This appendix focuses on technical prompts. The experiment prompts are shortened from templates saved in the project. The planning and coding prompts are reconstructed examples based on the team's description and the implemented design; they are not verified copies of earlier chats. AI also helped draft and revise the report, but report-writing prompts are not reproduced here.

A. Planning the Memory and Pipeline

We first defined the basic memory architecture and pruning rules. We then asked AI to review this design and suggest improvements. The prompts below are reconstructed examples of that type of request.

1) **Improving our memory design.** "Our memory stores linked needs with a status, confidence, supporting quotes and timestamps. Confidence fades over time. Weak beliefs become inactive, and very weak ones are pruned. Rejected needs and the change log are kept. We have defined this basic architecture and pruning process. Please review it and suggest improvements. Check whether pruning could remove useful information or break links between needs. Explain each suggested change and how we can test it."
2) **Connecting memory to the agents.** "Help us decide what the Analyzer should write to memory and what the Strategist, Generator and Critic should read. Keep the original conversation available. Show where to check evidence and how to stop a rejected guess from being repeated."

These examples follow the memory and agent interfaces in the implementation.

B. Coding and Result Calculation

Reconstructed examples.

1) **Memory updates and tests.** "Help us implement a Python memory module with a need, parent link, status, confidence, evidence quotes and timestamps. Keep a log of changes. Add focused tests for confirmation, rejected guesses, confidence decay and restoring saved memory. Follow the rules in our configuration."
2) **Checking the Analyzer output.** "Check that every evidence quote returned by the Analyzer really appears in the cited user turn. Flag unsupported claims and invalid links between needs. Suggest a small fix and test cases for these errors."
3) **Calculating final comparisons.** "Read our saved scores and calculate the differences between the specified systems. Use bootstrap samples over people, and use paired samples only when the experiment supports pairing. Apply Holm correction to the 42 planned comparisons. Report the values and intervals without filling in missing results."

The corresponding code is in the memory, grounding and statistical-analysis modules. Calculations use saved experiment outputs; an LLM is not asked to invent the result tables.

C. Prompts Used to Produce and Score Experiment Outputs

The following are plain-language summaries of saved prompts. The full templates also specify the input fields, rating items and output format.

1) **Finding needs from evidence.** "Read the conversation and list the person's likely feelings and needs. Support every claim with their exact words and give a confidence score. Mark a need as a guess, confirmed, rejected or resolved. Leave out claims with no supporting quote." Source: `prompts/agent_analyzer.md`.
2) **Annotating a supporter turn.** "For this reply, describe what is happening for the person and what strategy the supporter uses. Write specific phrases in simple words, not broad labels such as empathy. Return the analysis, strategy and disclosure-depth labels." Source: `prompts/annotate_turn.md`.
3) **Scoring intrusiveness.** "Score this supporter reply from 1 to 7 on each given item. Check for early advice, unsupported assumptions, probing too deeply and ignoring boundaries. Return only the item numbers and scores." Source: `prompts/metric_ip.md`.
4) **Scoring resistance.** "Read the user's reply after the supporter's message. Use the given 1-to-7 items to score irritation, disagreement, dismissal, defence of autonomy, deflection and withdrawal. Return only the item numbers and scores." Source: `prompts/metric_pri.md`.
5) **Scoring need identification.** "Compare the conversation with the person's reference need. Rate how clearly the supporter identified it using the given 1-to-7 scale. Count it only when the supporter puts the need into words; the user saying it alone does not count." Source: `prompts/scale_success.md`.

6) **Checking a stored need.** "Compare the need stored by the system with the reference needs. Score from 1 to 7 whether they describe the same underlying need, even if the wording differs." Source: `extra/prompts/memory_match.md`.

D. Models and Responsibility

The experiments use open-weight models, including Qwen2.5, Qwen3 and Mistral-Nemo, with ESC-RANK for dataset scoring. They use tools such as Ollama and Hugging Face Transformers. The experimental model roles are described in the methodology. Team responsibilities are listed in Appendix A; the authors remain responsible for the implementation and the reported claims.

APPENDIX C
ADDITIONAL EVALUATION DETAILS

The main results and ablation tables report the system comparisons. The full score summaries and the record of all 42 statistical tests are retained in the project's supplementary result files.

Success as a fraction. Counting a session as a success only when the judge rated it at least 5 out of 7 ("said the real need, at least partly"), every system reaches at most about 5% of sessions. The highest are the fine-tuned 7B with memory and the untuned four agents with no memory (both 0.048), and the lowest is the untuned model with the reactive user (0.000).